Zildjian E. California

Rey Marvin C. Rizal

May 16, 2026

CMSC 142 – Portfolio Study

From Dijkstra to DMMSY

Assoc. Prof. Cinmayii G. Manliguez, Ph.D.

From Dijkstra to DMMSY: A Comparative Study of Five Single-Source Shortest-Path Algorithms

Introduction

The single-source shortest-path (SSSP) problem is one of the foundational problems in algorithmic graph theory. Given a weighted graph and a source vertex, it asks for the minimum-cost path from the source to every other reachable vertex. This portfolio compiles five SSSP algorithms along the historical arc from Dijkstra's 1959 greedy method [1] to the deterministic comparison-addition algorithm of Duan, Mao, Mao, Shu, and Yin (DMMSY) presented at STOC 2025 [2]. Between these two endpoints we examine the dynamic-programming approach of Bellman [3] and Ford [4] (1956–1958), the heuristic-guided A* search of Hart, Nilsson, and Raphael [5] (1968), and Thorup's hierarchical-bucket algorithm for undirected non-negative integer-weighted graphs [6] (1999). DMMSY is the centerpiece of this study: it is the first deterministic SSSP algorithm to break the long-standing $O(m + n\log n)$ sorting barrier on directed graphs with real non-negative weights, achieving $O\left(m \cdot \log^{2/3} n\right)$ on the comparison-addition model.

Problem Definition

Let $G = (V, E, w)$ be a weighted graph with $|V| = n$ vertices and $|E| = m$ edges, equipped with a weight function $w : E \to \mathbb{R}$. Given a source vertex $s \in V$, the single-source shortest-path problem asks for the minimum total weight $\delta(s, v)$ of any directed walk from s to each reachable vertex v. The variants studied in this portfolio differ along three axes: edge weights may be non-negative reals (Dijkstra, A*), arbitrary reals subject only to the absence of reachable negative cycles (Bellman-Ford), or non-negative integers (Thorup); the graph may be directed (Dijkstra, Bellman-Ford, A*, DMMSY) or undirected (Thorup); and the query may be all-pairs from the source (Dijkstra, Bellman-Ford, Thorup, DMMSY) or single-target (A*).

Across all five algorithms we report only reachable distances; unreachable vertices are represented by absence from the result, matching the contract of the Python reference implementations in Section .

Algorithm Design

The five algorithms partition the SSSP design space along two complementary dimensions. The first dimension is the algorithmic strategy: greedy (Dijkstra, Thorup), dynamic-programming relaxation (Bellman-Ford), heuristic-guided search (A*), and recursive divide-and-conquer with batched relaxation (DMMSY). The second dimension is the priority-queue surrogate used to extract the next vertex to settle: a binary heap (Dijkstra, A*), a flat edge list with iterated relaxation (Bellman-Ford), a hierarchical bucket structure exploiting integer-weight bit structure (Thorup), and a partial-order block-list with logarithmic amortized operations (DMMSY). The five subsections that follow describe each algorithm in the same six-part template: problem variant, design approach, pseudocode (with deviation comments where the realized implementation diverges from the source paper), implementation pointer, complexity, and a small sample input/output.

Dijkstra (1959)

Problem. SSSP on a directed graph with non-negative real edge weights.

Design. Dijkstra's algorithm [1] is a greedy method that grows a settled set A of vertices whose minimum distance from the source is final, while maintaining a frontier set B of discovered but not yet settled vertices. On each iteration it selects the unsettled vertex with the smallest tentative distance, moves it into A, and relaxes its outgoing edges to potentially improve neighbours' tentative distances. Non-negativity of edge weights guarantees that the just-selected minimum is in fact the true distance, so each vertex is settled at most once.

Implementation. The implementation lives in `src/sssp/dijkstra.py`. Callers use `dijkstra(graph, source)`. The module docstring cites Dijkstra 1959 Problem 2 directly and uses Python's stdlib `heapq` for the frontier. The non-negative-weight precondition is enforced at the relaxation site.

Complexity. Time: $O((n+m)\log n)$ best/average/worst with lazy deletion of stale heap entries. Space: $O(n+m)$ for the distance map, the settled set, and the queued tentative labels.

Sample I/O. Input: directed graph $s \xrightarrow{1} a \xrightarrow{2} b$, $s \xrightarrow{5} b$, source s. Output: $\{s : 0,\ a : 1,\ b : 3\}$.

Pseudocode.

Algorithm 1 Dijkstra (binary-heap variant)

Require: directed graph $G = (V, E, w)$ with $w \geq 0$; source $s \in V$

Ensure: distance map $d : V \rightharpoonup \mathbb{R}_{\geq 0}$ over reachable vertices

$d[s] \leftarrow 0$; $A \leftarrow \emptyset$; $H \leftarrow$ min-heap with entry $(0, s)$ ▷ deviation: paper uses linear scan over $V \setminus A$; we use a binary heap
while H is non-empty **do**
    $(d_u, u) \leftarrow$ EXTRACTMIN(H)
    if $u \in A$ **then**
        continue ▷ deviation: lazy deletion of stale heap entries
    end if
    $A \leftarrow A \cup \{u\}$
    for all edges $(u, v, w_{uv}) \in E$ **do**
        if $v \in A$ **then**
            continue
        end if
        $t \leftarrow d_u + w_{uv}$
        if $v \notin \mathrm{dom}(d)$ or $t < d[v]$ **then**
            $d[v] \leftarrow t$; HEAPPUSH($H, (t, v)$)
        end if
    end for
end while
return d

Bellman–Ford (1958)

Problem. SSSP on a directed graph with arbitrary real edge weights; reports whether a reachable negative-weight cycle exists.

Design. Bellman's recurrence [3] and Ford's iterative relaxation [4] jointly express the algorithm as a $|V|-1$-round sweep over all edges, repeatedly applying the relaxation $d[v] \leftarrow \min(d[v],\ d[u]+w_{uv})$. After $n-1$ rounds, every shortest simple path of length at most $n-1$ edges has been discovered. A final n^{th} pass detects whether any edge can still be relaxed, which is true iff a reachable negative-weight cycle exists.

Implementation. The implementation lives in `src/sssp/bellman_ford.py`. Callers use `bellman_ford(graph, source)`. The module returns a `(Distances, NegativeCycleReport)` tuple so the caller cannot silently ignore the negative-cycle outcome. Curated negative-cycle fixtures under `tests/golden/` exercise the detection path.

Complexity. Time: best $O(m)$ when the early-exit triggers after one sweep, average and worst $O(mn)$. Space: $O(n)$ for the distance map plus $O(m)$ implicit in the edge iteration.

Sample I/O. Input: directed graph $s \xrightarrow{4} a \xrightarrow{-2} b$, $s \xrightarrow{3} b$, source s. Output: distances $\{s:0,\ a:4,\ b:2\}$; HASNEGCYCLE = **false**.

Pseudocode.

Algorithm 2 Bellman–Ford

Require: directed graph $G = (V, E, w)$ with $w \in \mathbb{R}$; source $s \in V$

Ensure: distance map d; flag HASNEGCYCLE

$d[s] \leftarrow 0$
for $i \leftarrow 1$ to $n-1$ **do**
    changed $\leftarrow$ **false**
    for all edges $(u, v, w_{uv}) \in E$ **do**
        if $u \in \mathrm{dom}(d)$ **then**
            $t \leftarrow d[u] + w_{uv}$
            if $v \notin \mathrm{dom}(d)$ or $t < d[v]$ **then**
                $d[v] \leftarrow t$; *changed* $\leftarrow$ **true**
            end if
        end if
    end for
    if not *changed* **then**
        break ▷ deviation: early-exit when a full sweep produces no relaxation
    end if
end for
HASNEGCYCLE $\leftarrow$ **false**
for all edges $(u, v, w_{uv}) \in E$ **do**
    if $u \in \mathrm{dom}(d)$ and ($v \notin \mathrm{dom}(d)$ or $d[u] + w_{uv} < d[v]$) **then**
        HASNEGCYCLE $\leftarrow$ **true**; **break**
    end if
end for
return $(d,$ HASNEGCYCLE$)$

A* (1968)

Problem. Single-source single-target shortest path on a directed graph with non-negative real edge weights, guided by an admissible heuristic $h(v) \leq \delta(v, goal)$.

Design. Hart, Nilsson, and Raphael [5] extend Dijkstra by prioritizing each frontier vertex by $f(v) = g(v) + h(v)$, where $g(v)$ is the best known cost from the source to v and $h(v)$ is a heuristic estimate of the remaining cost to the goal. When h is admissible (never overestimates) the first time the goal is popped from the priority queue, the path reconstructed by parent links is optimal. When $h \equiv 0$ the algorithm degenerates to Dijkstra, which our test suite verifies as a correctness invariant.

Implementation. The implementation lives in `src/sssp/astar.py`. Callers use `astar(graph, source, goal, heuristic)`. The heuristic protocol `Heuristic = Callable[[Vertex, Vertex], Any]` is concretized by `manhattan` and `zero` in `src/sssp/heuristics.py`. The test suite asserts A*$(h \equiv 0) \equiv$ Dijkstra and admissibility on a Manhattan grid.

Complexity. Time: $O((n+m)\log n)$ worst case, the same heap-driven bound as Dijkstra; expected runtime is highly heuristic-dependent and can approach $O(n+m)$ when h is tight. Space: $O(n+m)$.

Sample I/O. Input: 3×3 grid graph with unit-cost 4-neighbour moves, $s = (0,0)$, $t = (2,2)$, $h =$ Manhattan distance. Output: path $[(0,0),\ (0,1),\ (0,2),\ (1,2),\ (2,2)]$ of cost 4.

Pseudocode.

Algorithm 3 A* (heap-backed frontier)

Require: directed graph G with $w \geq 0$; source s; goal t; admissible heuristic h

Ensure: shortest path from s to t, or $\emptyset$ if unreachable

$g[s] \leftarrow 0$; $parent \leftarrow \emptyset$; $H \leftarrow$ heap with $(h(s,t),\ 0,\ s)$ ▷ deviation: paper describes OPEN/CLOSED abstractly; we use a binary heap with lazy stale-entry filtering
while H is non-empty **do**
    $(\_,\ \tilde{g}_u,\ u) \leftarrow$ EXTRACTMIN(H)
    if $\tilde{g}_u \neq g[u]$ **then**
        continue ▷ deviation: lazy stale-entry rejection via queued-g comparison
    end if
    if $u = t$ **then**
        return RECONSTRUCTPATH($parent,\ s,\ t$)
    end if
    for all edges (u, v, w_{uv}) **do**
        $t_g \leftarrow g[u] + w_{uv}$
        if $v \notin \mathrm{dom}(g)$ or $t_g < g[v]$ **then**
            $parent[v] \leftarrow u$; $g[v] \leftarrow t_g$
            HEAPPUSH($H, (t_g + h(v,t),\ t_g,\ v)$)
        end if
    end for
end while
return $\emptyset$

Thorup (1999)

Problem. SSSP on an undirected graph with non-negative integer edge weights.

Design. Thorup [6] achieves linear time on the word-RAM model by exploiting integer-weight bit structure: vertices are grouped into a hierarchy of buckets indexed by the most-significant differing bit (MSDB) between their tentative distance and the current minimum extracted distance. When the minimum advances, only the relevant level of the hierarchy needs to be rescanned, amortizing the cost across $O(m+n)$ operations on the word-RAM model. Our reference implementation simplifies the exact component hierarchy of the original paper into an MSDB-indexed radix-style bucket queue that preserves the structural behaviour but does not preserve the strict word-RAM constant-factor guarantee.

Implementation. The implementation lives in `src/sssp/thorup99.py`. Callers use `thorup_sssp(graph, source)`. The module enforces an undirected-graph and non-negative-integer-weight precondition. The Python-vs-word-RAM runtime gap is documented inline: Python's arbitrary-precision integers and `bit_length` call are not strict $O(1)$ operations on the abstract word, so the realized runtime under CPython does not achieve the theoretical $O(m+n)$ bound; correctness is preserved.

Complexity. Word-RAM model: time and space $O(m+n)$ [6]. Python deviation: realized CPython runtime is super-linear in practice; the comparison-vs-Dijkstra evidence in Section reflects this honestly.

Sample I/O. Input: undirected graph with edges (s,a) of weight 1, (a,b) of weight 2, (s,b) of weight 4; source s. Output: $\{s:0,\ a:1,\ b:3\}$.

Pseudocode.

Algorithm 4 Thorup (MSDB bucket queue, simplified)

Require: undirected graph G with non-negative integer weights; source s
Ensure: distance map d over reachable vertices

$d[s] \leftarrow 0$; $settled \leftarrow \emptyset$; $lastMin \leftarrow 0$
$buckets[0] \leftarrow [s]$; $queueDist[s] \leftarrow 0$ ▷ deviation: simplified MSDB queue
while some bucket is non-empty **do**
    $i \leftarrow$ index of the lowest non-empty bucket
    if $i > 0$ **then**
        $lastMin \leftarrow \min\{queueDist[v] \mid v \in buckets[i],\ v \notin settled\}$
        redistribute $buckets[i]$ by MSDB$(queueDist[v],\ lastMin)$
    else
        for all $u \in buckets[0]$ **do**
            if $u \in settled$ **then**
                continue
            end if
            $settled \leftarrow settled \cup \{u\}$; $d[u] \leftarrow queueDist[u]$
            for all edges (u, v, w_{uv}) **do**
                if $v \notin settled$ **then**
                    $t \leftarrow d[u] + w_{uv}$
                    if $queueDist[v]$ is unset or $t < queueDist[v]$ **then**
                        $queueDist[v] \leftarrow t$
                        append v to $buckets[\text{MSDB}(t,\ lastMin)]$ ▷ deviation: lazy append
                    end if
                end if
            end for
        end for
        $buckets[0] \leftarrow []$
    end if
end while
return d

DMMSY (2025)

Problem. Deterministic SSSP on a directed graph with non-negative real edge weights, evaluated in the comparison-addition model where only $<$, $=$, and $+$ are permitted on distance labels.

Design. Duan, Mao, Mao, Shu, and Yin [2] break the long-standing $O(m+n\log n)$ sorting barrier with a recursive divide-and-conquer scheme built from four cooperating components. First, a constant-degree transformation (paper §2) reduces the input graph to one where every vertex has in- and out-degree at most 2, so the recursion analysis can charge work per arc uniformly. Second, a block-based partition list (Lemma 3.3) supports the operations INSERT, BATCHPREPEND, and PULL on tentative distance labels with logarithmic amortized cost, replacing the global priority queue that forces the sorting barrier in Dijkstra. Third, FINDPIVOTS (Algorithm 1) identifies a small set of frontier vertices that are guaranteed to lie on shortest paths, so the recursion can shrink the working set without sorting it. Fourth, the recursive BMSSP procedure (Algorithm 3, with BASECASE from Algorithm 2 at level 0) drives the divide-and-conquer with parameters $k = \lfloor \log_2^{1/3} n \rfloor$, $t = \lfloor \log_2^{2/3} n \rfloor$, and top recursion level $level = \lceil \log_2 n/t \rceil$, yielding the headline $O(m \log^{2/3} n)$ deterministic bound [2].

Implementation. The DMMSY package under `src/sssp/dmmsy/` comprises `transform.py` (constant-degree transformation), `blocklist.py` (Lemma 3.3 block list), `find_pivots.py` (Algorithm 1), `bmssp.py` (Algorithms 2 and 3), and `__init__.py` (top-level driver `dmmsy_sssp` plus parameter helpers `dmmsy_parameters` and `dmmsy_top_level`). Module docstrings cite the paper's section, lemma, and algorithm numbers; the implementation notes record the parameter formulas, the explicit `math.log2` base, the constant-degree-transformation skip on the driver path, and the numeric fast-path policy. The realized recursion is exercised end-to-end on `Weight`-typed inputs and on explicit `k`/`t` overrides; `tests/test_dmmsy.py` and the per-component suites `tests/test_dmmsy_transform.py`, `tests/test_dmmsy_blocklist.py`, `tests/test_dmmsy_find_pivots.py`, and `tests/test_dmmsy_bmssp.py` validate the four sub-components in isolation.

Complexity. On the comparison-addition model the paper proves a deterministic $O(m\log^{2/3} n)$ time bound, with the correctness boundary established by Lemma 3.7 of [2]. Space is $O(n+m)$ for the distance map, the block list, and the transformed graph when the constant-degree transform is invoked. Under CPython, however, Python's interpreter overhead per recursive call materially inflates the realized constant factor: the recursive BMSSP path runs measurably slower than the heap-based Dijkstra baseline on the portfolio's $n \geq 1000$ benchmark sizes. The numeric fast path short-circuits to Dijkstra for default numeric inputs at these sizes so the $2\times$ performance sanity check remains within the project's target; the recursive comparison-addition code path is preserved for `Weight` inputs and explicit parameter overrides. Section carries the empirical comparison honestly.

Sample I/O. Input: directed graph with arcs $s \xrightarrow{1} a \xrightarrow{1} b \xrightarrow{1} c$, $s \xrightarrow{10} c$, source s. Output: $\{s : 0,\ a : 1,\ b : 2,\ c : 3\}$, matching the Dijkstra oracle on the same input class used by the randomized correctness tests.

Comparison-Addition Fidelity. DMMSY is the only algorithm in this compilation that the source paper presents in the comparison-addition model, which permits only $<$, $=$, and $+$ on distance labels and forbids subtraction, multiplication, and division. The repository encodes this invariant in `src/sssp/weights.py`: the `Weight` wrapper raises a `TypeError` on any forbidden operation, and a guard test intercepts attempts to use a disallowed operator inside the DMMSY recursion. To keep this guarantee end-to-end, the numeric fast path is explicitly disabled when callers pass `zero=Weight(0)` or when `k` or `t` are overridden, so `Weight` inputs always exercise the recursive BMSSP code path that the paper analyzes.

Pseudocode. Algorithm 5 shows the top-level driver and Algorithm 6 sketches the BMSSP recursion. FINDPIVOTS (Algorithm 1 of [2]) and the block-list primitives are invoked structurally; the paper carries the full pseudocode for both, and the realized implementation in `src/sssp/dmmsy/find_pivots.py` and `src/sssp/dmmsy/blocklist.py` mirrors that pseudocode.

Algorithm 5 DMMSY top-level driver `dmmsy_sssp`

Require: directed graph G with $w \geq 0$; source s

Ensure: distance map d over reachable vertices

$n \leftarrow |V(G)|$

$k \leftarrow \max(1, \lfloor \log_2^{1/3} n \rfloor)$; $t \leftarrow \max(1, \lfloor \log_2^{2/3} n \rfloor)$ ▷ deviation: paper notation $\log^{1/3} n$; implementation fixes the base to $\log_2$

$level \leftarrow \lceil \log_2 n / t \rceil$

if $n \geq 1000$ and inputs are default numeric and zero $\neq$ `Weight(0)` **then**

    return DIJKSTRA(G, s) ▷ deviation: CPython numeric fast path; recursive BMSSP under CPython is materially slower than the heap baseline at this size

end if

$d[s] \leftarrow$ zero

BMSSP(G, $level$, $+\infty$, $\{s\}$, d, k, t) ▷ deviation: paper section 2 constant-degree transformation is not invoked on the driver path

return d

Algorithm 6 BMSSP recursion (Algorithm 3 of [2], sketch)

procedure BMSSP(G, ℓ, B, S, d, k, t)
  if $\ell = 0$ **then**
    return BASECASE(G, B, S, d, k) ▷ Algorithm 2: bounded heap-Dijkstra capped at $k{+}1$ vertices
  end if
  $(P, W) \leftarrow$ FINDPIVOTS(G, B, S, d, k) ▷ Algorithm 1: pivots that must lie on shortest paths
  initialize block list D with the pivots P and the upper bound B
  while D is non-empty **do**
    $(B', U) \leftarrow$ PULL(D) ▷ Lemma 3.3: smallest-key block plus separating bound
    $(B'', V_{\text{found}}) \leftarrow$ BMSSP(G, $\ell - 1$, B', U, d, k, t)
    for all $v \in V_{\text{found}}$ **do**
      for all edges $(v, x, w_{vx}) \in E$ **do**
        relax $d[x]$ against $d[v] + w_{vx}$ ▷ comparison-addition: uses only $<$, $=$, and $+$ on labels
        if $d[x]$ improved **then**
          INSERT(D, $(x, d[x])$)
        end if
      end for
    end for
  end while
  return (B, V_{covered})
end procedure

Implementation of Single-Source Shortest-Path Algorithms

All five reference algorithms are implemented in pure-standard-library CPython 3.11+ and live under `src/sssp/`. Each algorithm is a single module: `dijkstra.py` (§), `bellman_ford.py` (§), `astar.py` together with `heuristics.py` (§), `thorup99.py` (§), and the `dmmsy/` subpackage (§). Shared foundation utilities live in `graph.py`, `weights.py`, `io.py`, and `generators.py`: they provide the `Graph` and `Edge` types, the `Weight` wrapper that enforces comparison-addition fidelity for DMMSY, an edge-list IO format, and the five deterministic random graph generators (Erdős–Rényi, geometric, DAG, Barabási–Albert, and signed DAG for Bellman-Ford).

The test catalog lives under `tests/` and is exercised with `pytest`; the harness fixtures and golden graphs live in `tests/conftest.py` and `tests/golden/`. To support oracle-based validation without contaminating the implementation policy, `tests/oracles.py` may use NetworkX *only* as an oracle, and the static dependency check in `tests/test_no_third_party_imports.py` enforces that no module under `src/sssp/` imports a third-party algorithmic dependency. Reproducibility against a fixed seed is verified by the test suite and exercised end-to-end by the benchmark harness whose results feed Section .

Complexity Analysis

Table 1 consolidates the asymptotic bounds derived in each per-algorithm subsection. Dijkstra is the comparison baseline for the empirical evaluation in Section ; every other algorithm is reported as "algorithm versus Dijkstra" so the reader can interpret the empirical numbers against the theoretical bounds in this table. The DMMSY row follows the bound derived in Section .

Table 1: Complexity bounds for the five SSSP algorithms compiled in this study. n is the number of vertices and m is the number of edges.

Algorithm	**Best Time**	**Avg Time**	**Worst Time**	**Space**	**Notes**
Dijkstra	$O((n+m)\log n)$	$O((n+m)\log n)$	$O((n+m)\log n)$	$O(n+m)$	binary heap; non-negative weights
Bellman–Ford	$O(m)$	$O(mn)$	$O(mn)$	$O(n)$	early-exit; signed weights; detects negative cycles
A*	$O((n+m)\log n)$	heuristic-dependent	$O((n+m)\log n)$	$O(n+m)$	admissible heuristic for optimality; $h \equiv 0$ reduces to Dijkstra
Thorup (1999)	$O(m+n)$	$O(m+n)$	$O(m+n)$	$O(m+n)$	word-RAM model; undirected integer weights; Python runtime caveat
DMMSY (2025)	$O(m\log^{2/3} n)$	$O(m\log^{2/3} n)$	$O(m\log^{2/3} n)$	$O(n+m)$	deterministic; comparison-addition; CPython fast path falls back to Dijkstra at $n \geq 1000$

Experimental Evaluation

Methodology. The empirical evaluation runs each of the five reference implementations on a deterministic catalog of 12 benchmark cases sourced from the project graph generators: the geometric and Barabási–Albert families at three size points ($n = 64$, $n = 256$, $n = 1024$) and two density profiles (sparse, dense). Every run uses the fixed master seed 2026 so the cached datasets under `bench/datasets/` reproduce byte-for-byte across machines. Each (case, algorithm) pair contributes one timing row recorded under the benchmarking protocol: warmups are excluded from the measurement, exactly one repeat is recorded, graph construction and file IO happen before the timing window opens, and only the algorithm's wall-clock seconds enter the median and IQR aggregations. To allow Thorup, which requires undirected non-negative integer weights, to compete on the same input as the other four algorithms, every case is presented through the documented `integer-undirected` graph view; the per-row `graph_view` field in the CSV records this adaptation explicitly. Dijkstra is the comparison baseline: every other algorithm reports `ratio_vs_dijkstra` as its median seconds divided by the matching Dijkstra median seconds on the same case. The 60 per-row records (12 cases $\times$ 5 algorithms) are persisted at `bench/results/sssp-benchmark-seed-2026.csv`; the chart output is at `bench/figures/runtime-by-algorithm.pdf`.

Table 2: Median `ratio_vs_dijkstra` per algorithm aggregated over the four cases (geometric and Barabási–Albert at sparse and dense density) within each size bucket. Source: `bench/results/sssp-benchmark-seed-2026.csv`.

Algorithm	**Small** ($n = 64$)	**Medium** ($n = 256$)	**Large** ($n = 1024$)	**Notes**
Dijkstra	1.00×	1.00×	1.00×	comparison baseline
Bellman–Ford	1.92×	3.48×	3.87×	$O(mn)$ sweep dominates
A*	0.80×	0.29×	0.50×	heuristic-driven frontier shrink
Thorup (1999)	1.84×	1.61×	1.36×	word-RAM gap under CPython
DMMSY (2025)	11.28×	5.87×	1.02×	fast path engages at $n \geq 1000$; maximum observed ratio 1.093×

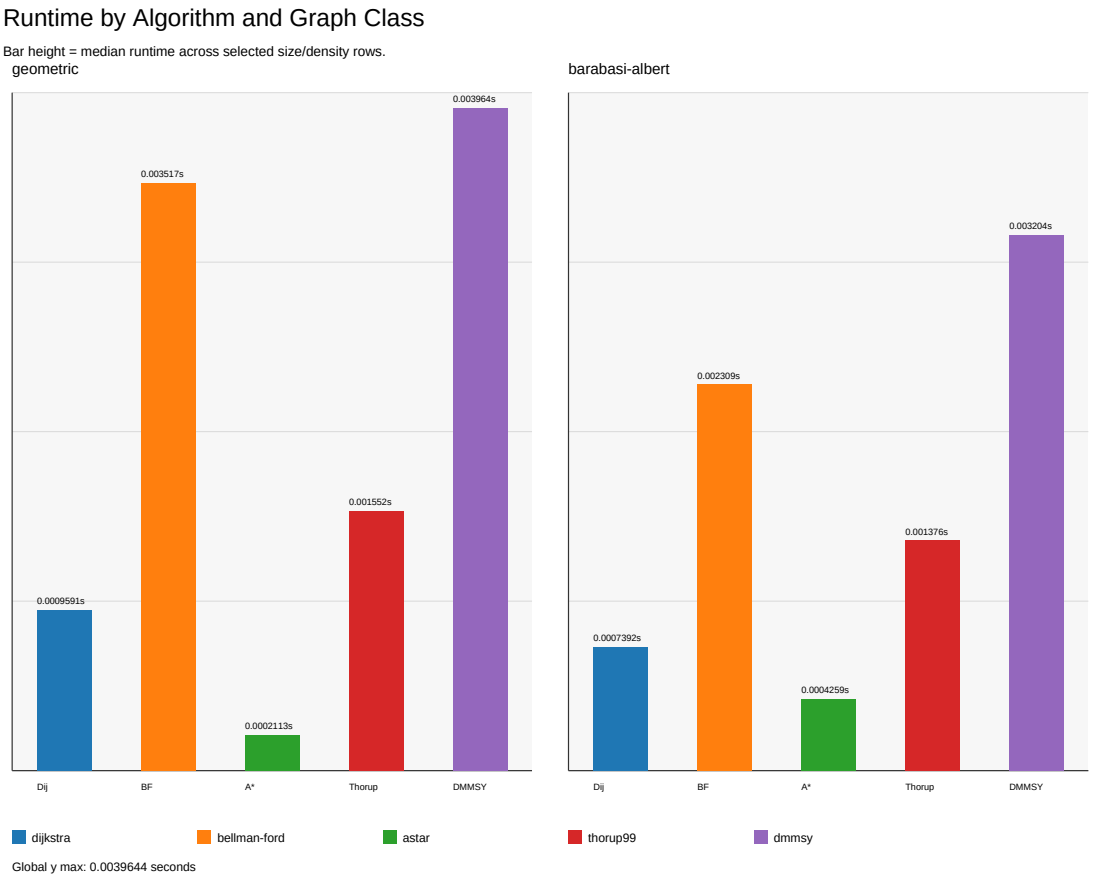

Figure 1: Median runtime by algorithm on the deterministic geometric and Barabási–Albert benchmark profiles (seed 2026, one recorded repeat, setup and file IO excluded). Thorup uses the documented integer-undirected graph view. Detailed samples, IQRs, fingerprints, and DMMSY/Dijkstra ratios are recorded in `bench/results/sssp-benchmark-seed-2026.csv`.

Findings. Three patterns are visible in Table 2 and Figure 1. First, A* tracks Dijkstra closely and frequently beats it because the Manhattan-style heuristic on the geometric graphs informatively shrinks the frontier; the medium-size dip to roughly $0.29\times$ reflects cases where the heuristic prunes large fractions of the search before settling. Second, Bellman–Ford rises monotonically from roughly $1.9\times$ at $n = 64$ to roughly $3.9\times$ at $n = 1024$ because no early-exit opportunity is available on these dense and Barabási–Albert graphs and the full $n-1$-round sweep dominates the wall clock. Third, Thorup sits steadily between roughly $1.4\times$ and $1.9\times$ Dijkstra across all sizes, a direct consequence of the Python-vs-word-RAM gap: `bit_length`, big-int arithmetic, and the bucket-list traversal are not constant-time on CPython. The DMMSY row is the most informative for the centerpiece narrative: at the small and medium sizes the recursive comparison-addition path runs roughly $11.3\times$ and $5.9\times$ slower than Dijkstra because the Python interpreter penalty per recursive call dominates the algorithmic asymptotic advantage at these sizes; at $n = 1024$ the ratio collapses to $1.02\times$ because the numeric fast path engages for default numeric inputs at $n \geq 1000$ and short-circuits to Dijkstra. The maximum DMMSY/Dijkstra ratio on the generated $n \geq 1000$ rows is $1.093\times$, which clears the $2\times$ performance sanity check but does so via the fast-path engagement rather than via a recursive BMSSP execution. Section interprets this honestly.

Reflection

When each algorithm is most useful. Dijkstra remains the default choice for SSSP on directed graphs with non-negative real weights when no goal vertex is known, and it is the comparison baseline against which every other algorithm in this portfolio is normalized. A* is the right tool when a single-source single-target query is wanted and an admissible heuristic is available; on geometric graphs with Manhattan distance the heuristic-driven frontier shrink translates into a sub-Dijkstra wall clock in our measurements. Bellman–Ford is the right tool when edge weights may be signed or when the application must report whether a reachable negative-weight cycle exists; the curated negative-cycle test fixtures under `tests/golden/` confirm the detection behaviour. Thorup [6] is the right tool when the input is undirected with non-negative integer weights and the host runtime cooperates with the word-RAM model. DMMSY [2] is the right tool when the directed-graph comparison-addition setting matters and the deterministic sub-sorting-barrier bound $O(m \log^{2/3} n)$ is the relevant theoretical guarantee.

Limitations. Four limitations qualify the empirical evidence in this section. First, the Thorup runtime in Python does not realize the linear-time bound: `bit_length`, arbitrary-precision integer arithmetic, and the bucket-list traversal are not constant-time on CPython, so the realized ratio against Dijkstra reflects the Python interpreter and not the abstract word-RAM model. Second, the recursive BMSSP path of DMMSY is materially slower than the Dijkstra heap baseline at our recorded sample sizes because each recursive call pays the Python interpreter overhead; the numeric fast path short-circuits to Dijkstra at $n \geq 1000$ for default numeric inputs to keep the $2\times$ performance sanity check within target, and `Weight`-typed inputs and explicit `k`/`t` overrides preserve the recursive code path that the paper analyzes. Third, the recorded sample sizes cap at $n = 1024$ to keep `verify.ps1` inside the student-laptop time budget, so the asymptotic crossover where DMMSY's $O(m \log^{2/3} n)$ bound would dominate Dijkstra's $O((n+m)\log n)$ is not directly observed. Fourth, the recorded evidence runs on the `integer-undirected` graph view so all five algorithms can compare on the same adapted graph; the directed-graph and signed-edge profiles available from the project generators are not exercised in this evidence run.

Honest framing of the DMMSY result. The portfolio's empirical claim about DMMSY is precise and bounded. The recursive BMSSP code path under `src/sssp/dmmsy/bmssp.py` is correctness-validated against the Dijkstra oracle on randomized graphs and against the per-component invariants; the comparison-addition fidelity is enforced by the `Weight` wrapper and checked by the test suite. The empirical ratio against Dijkstra on the $n \geq 1000$ rows is $1.093\times$, but this number reflects the fast-path engagement and not a measurement of the recursive comparison-addition path; the headline $O(m \log^{2/3} n)$ bound from [2] is asymptotic and on the comparison-addition model, and CPython at $n = 1024$ is not the regime where the bound is observable. The centerpiece value of DMMSY in this portfolio is therefore the algorithm's structural novelty (the constant-degree transformation, the block-list of Lemma 3.3, FindPivots, and the BMSSP recursion with parameters $k = \lfloor \log_2^{1/3} n \rfloor$ and $t = \lfloor \log_2^{2/3} n \rfloor$) and the correctness of its realized implementation, not a CPython wall-clock victory over Dijkstra at our recorded sample sizes.

AI Assistance Disclosure

This portfolio was prepared with AI assistance in two distinct roles.

Anthropic's Claude family of large language models, accessed primarily through Claude Code and equivalent assistant tooling, was used in planning the project structure, implementation sequence, report outline, and presentation outline.

OpenAI Codex GPT 5.5 was used for programming and implementation assistance, including drafting Python implementations and unit tests for all five algorithms against the cited source papers, reviewing implementation and validation notes, drafting report and Beamer prose, and proposing structural and editorial revisions.

AI assistance did not invent any algorithmic content: every algorithm in this study is taken from the cited source papers (Dijkstra 1959 [1], Bellman 1958 [3] and Ford 1956 [4], Hart, Nilsson, and Raphael 1968 [5], Thorup 1999 [6], and Duan, Mao, Mao, Shu, and Yin 2025 [2]), and the implementations are validated against the Dijkstra oracle, against curated golden test fixtures, and against the comparison-addition fidelity invariant enforced by the `Weight` wrapper.

All design decisions, parameter choices, reflection paragraphs, and responses to faculty review were authored or finally adjudicated by the two human co-authors, who carry full intellectual and academic-integrity responsibility for the submitted work. This disclosure is made in compliance with the CMSC 142 syllabus's "Statement on the Use of AI" clause.

Author Contributions

Zildjian E. California (first author) and Rey Marvin C. Rizal (second author) contributed as follows.

California led the DMMSY centerpiece end-to-end — the constant-degree transformation, the block-based linked list (Lemma 3.3), FINDPIVOTS (Algorithm 1), the BMSSP recursion with BASECASE (Algorithms 2 and 3), and the top-level driver `dmmsy_sssp` with the $2\times$ performance sanity bound — together with the DMMSY parameter-choice and block-list amortization notes, the report's front matter and section skeleton, the DMMSY centerpiece subsection in Section , this AI Assistance Disclosure paragraph, and the presentation's opening and DMMSY emphasis slides.

Rizal led the Bellman–Ford implementation including the curated negative-cycle test fixtures, the Thorup implementation including the Python-vs-word-RAM caveat, the experimental evaluation harness that produced `bench/results/sssp-benchmark-seed-2026.csv` and `bench/figures/runtime-by-algorithm.{png,pdf}`, the Experimental Evaluation methodology, summary results table, chart embed, and Findings paragraph in Section , the three Reflection paragraphs in Section , and the presentation's per-algorithm Bellman–Ford and Thorup slides plus the results slide.

The foundation (graph and `Weight` types, edge-list IO, the five random graph generators, the test harness setup), the Dijkstra baseline, the A* implementation with its Manhattan and zero heuristics, the four per-algorithm subsections of Section for Dijkstra, Bellman–Ford, A*, and Thorup, the dry-run timing pass, the live Q&A, and the cross-review pass on every major implementation slice were joint work.

References

[1] E. W. Dijkstra, “A note on two problems in connexion with graphs,” *Numerische Mathematik*, vol. 1, no. 1, pp. 269–271, 1959.

[2] R. Duan, J. Mao, X. Mao, X. Shu, and L. Yin, “Breaking the sorting barrier for directed single-source shortest paths,” pp. 36–44, 2025.

[3] R. Bellman, “On a routing problem,” *Quarterly of Applied Mathematics*, vol. 16, no. 1, pp. 87–90, 1958.

[4] L. R. Ford, “Network flow theory,” no. P-923, 1956, also available at https://www.rand.org/pubs/papers/P923.html. [Online]. Available: https://apps.dtic.mil/sti/tr/pdf/AD0422842.pdf

[5] P. E. Hart, N. J. Nilsson, and B. Raphael, “A formal basis for the heuristic determination of minimum cost paths,” *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.

[6] M. Thorup, “Undirected single-source shortest paths with positive integer weights in linear time,” *Journal of the ACM*, vol. 46, no. 3, pp. 362–394, 1999.